CS 185/285 Assignment 3: Q-Learning and Actor-Critic — Report

All experiments ran locally with wandb in offline mode (`WANDB_MODE=offline`), so the training commands match the assignment handout. Every submitted run uses the provided configuration file unchanged; the runs use seed 1 unless stated otherwise. In every plot the x-axis is the number of environment steps. The figures are produced by `uv run src/scripts/plot.py`.

The runs required by the handout are in `exp/` (and in `submit.zip`). Additional runs that are only used in this report (learning-rate sweep, other Hopper seeds, sanity checks) are in `exp_extra/`.

Summary

Experiment	Handout target	Result	Met
DQN, CartPole-v1	eval return 500	500 (first reached at 72.5K steps)	yes
Double DQN, LunarLander-v2	200 at least once	maximum 256, final 222	yes
Double DQN, MsPacman	about 1500	maximum 2232, final 1438	yes
SAC, HalfCheetah-v4 (fixed temperature)	at least 6000	maximum 10,355, final 10,212	yes
SAC, HalfCheetah-v4 (auto-tuned temperature)	—	maximum 9549, final 9425	—
SAC clipped double-Q, Hopper-v4	1500 at least once	maximum 3474 (seed 3; first ≥ 1500 at 105K steps); seeds 1 and 2 reach 1034 and 1464	yes (1 of 3 seeds)

SAC was first checked on `InvertedPendulum-v4` with both sanity configurations (fixed and auto-tuned temperature); both reach the maximum return of 1000.

2. DQN

2.4 CartPole-v1

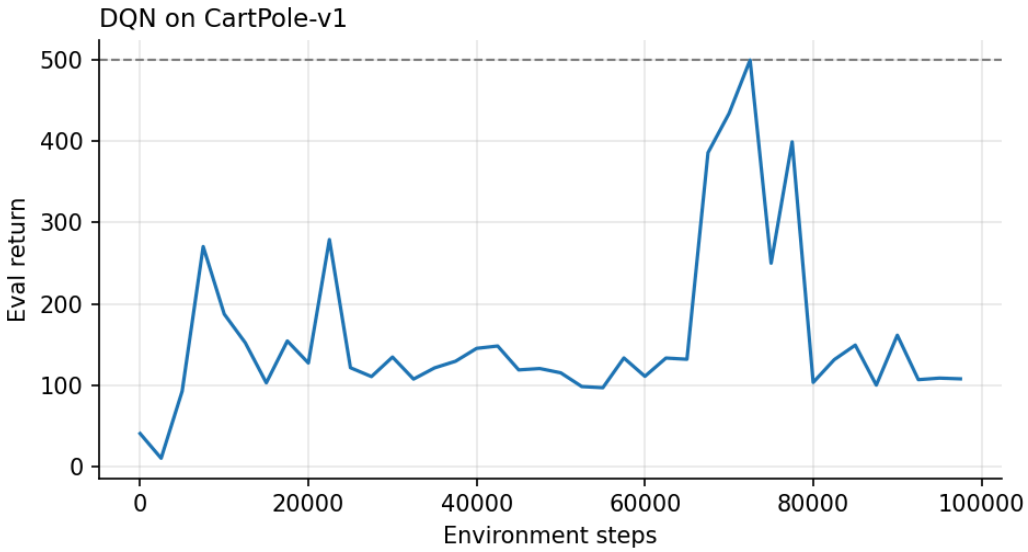


Figure 1: DQN on CartPole-v1

The eval return first reaches the maximum of 500 at 72.5K steps. The policy is not stable afterwards (the last evaluation is 108), which is common for DQN on CartPole with a constant learning rate: the Q-function keeps changing after the task is solved and the greedy policy sometimes degrades.

2.5 LunarLander-v2 (double DQN)

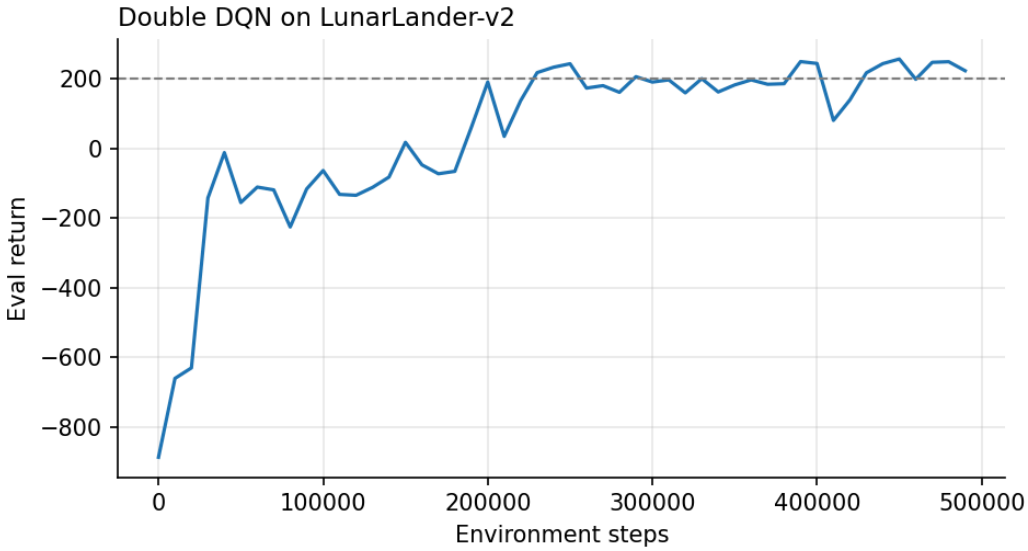


Figure 2: Double DQN on LunarLander-v2

The eval return first exceeds 200 at 230K steps, peaks at 256 and is 222 at the end of training (500K steps).

2.5 MsPacman (double DQN): train versus eval return

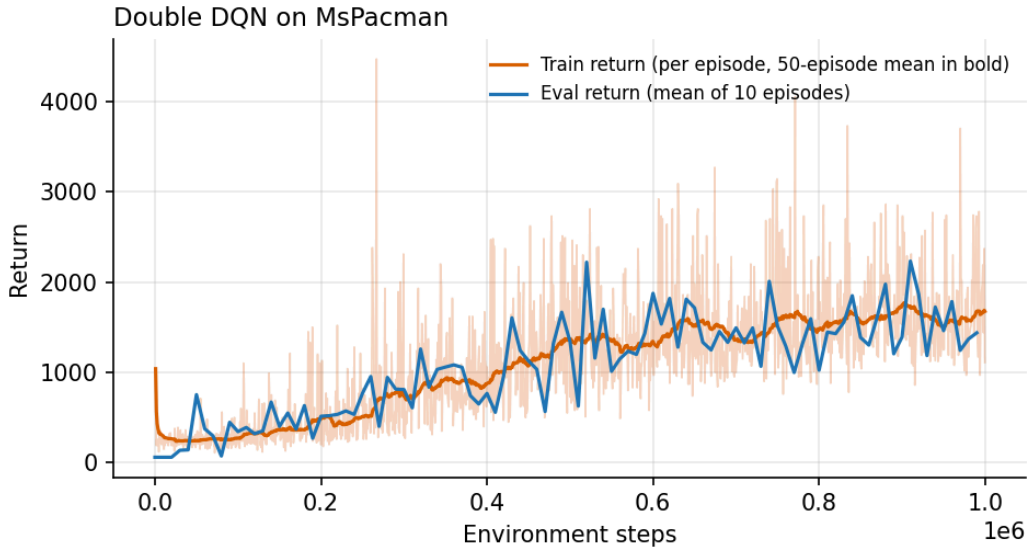


Figure 3: Double DQN on MsPacman

The eval return first exceeds 1500 at 430K steps and peaks at 2232 at 910K steps (final value 1438).

Why do the two curves look different early in training? The two returns are collected by different policies. Training episodes are collected with the ε-greedy policy, and ε starts at 1 and is annealed to 0.01 over roughly the first 500K steps, so early training episodes are essentially uniformly random play. Evaluation uses the greedy policy of the current Q-network. At the beginning the Q-network is untrained, and its greedy policy is close to a constant action: for the first 20K steps the eval return is exactly 60 in every evaluation, while the random training policy gets about 250, because random movement at least collects the pellets around the start position while a constant action runs into a wall and stops scoring. Early in training, the training return is therefore *higher* than the eval return. Once learning starts (after the first 20K steps) the Q-function quickly becomes informative and the order reverses: between 20K and 300K steps the eval return is on average somewhat higher than the training return (292 versus 257 over 20K–100K steps, 556 versus 490 over 100K–300K), because the training policy still takes a large fraction of random actions. As ε approaches its final value of 0.01 the two

policies become almost the same and the curves coincide (1570 versus 1620 over the last 100K steps). The training curve is also a per-episode value and is therefore much noisier than the eval curve, which averages 10 episodes.

2.6 Hyperparameter sweep: learning rate on LunarLander-v2

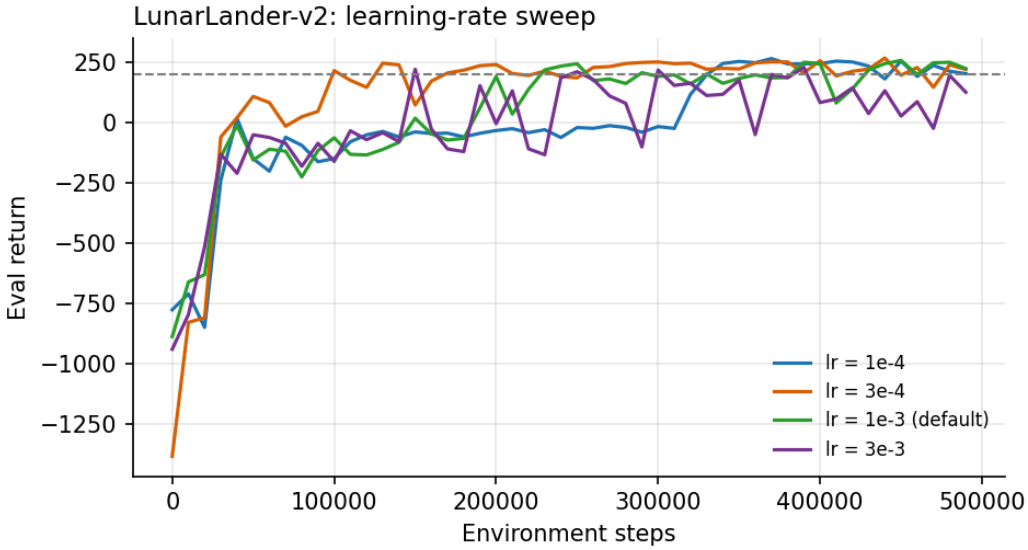


Figure 4: LunarLander-v2 learning-rate sweep

Caption. Eval return of double DQN on LunarLander-v2 for four learning rates: 1e-4, 3e-4, 1e-3 (the default) and 3e-3; everything else is unchanged. I chose the learning rate because it controls the basic trade-off of Q-learning with bootstrapped targets: a small step size makes the regression towards the (moving) targets slow, while a large step size makes the Q-function change quickly, which changes the targets and the greedy policy in turn and can make training unstable. It is also the hyperparameter that most often has to be re-tuned for a new environment.

Learning rate	First eval ≥ 200	Maximum	Final (500K steps)
1e-4	340K steps	264	201
3e-4	100K steps	266	219
1e-3 (default)	230K steps	256	222
3e-3	150K steps	228	124

The result follows the expected pattern. The smallest learning rate (1e-4) learns the slowest and needs 340K steps to reach 200. The largest (3e-3) reaches 200 at 150K steps but does not stay there: its last ten evaluations average 89, and the curve shows repeated collapses. The two middle values are both good; 3e-4 is the best of the four, reaching 200 after only 100K steps and staying above 200 in 30 of the 50 evaluations. These are single-seed results, so the difference between 3e-4 and 1e-3 should be read with caution, but the degradation at both ends of the range is clear.

3. Soft Actor-Critic

3.4 HalfCheetah-v4 with a fixed temperature

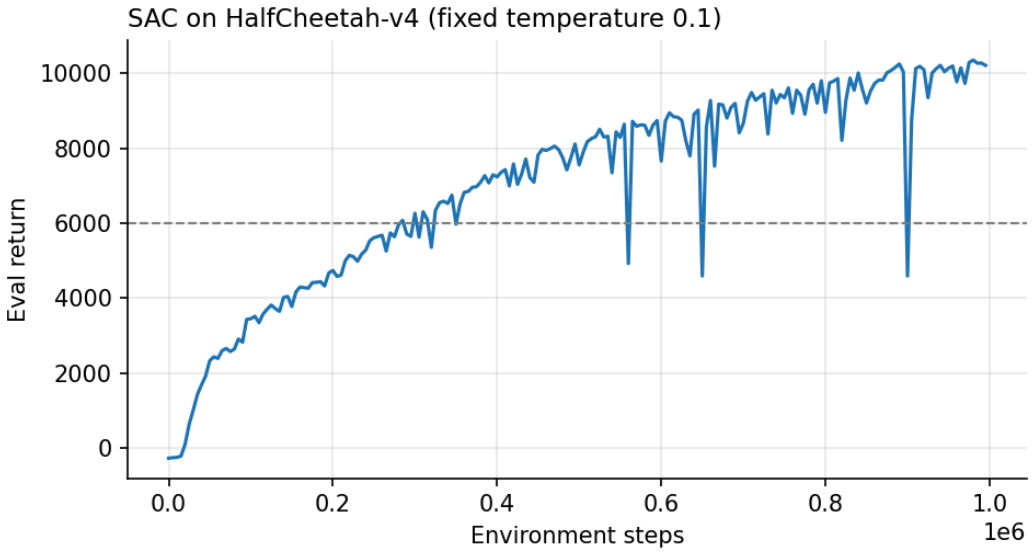


Figure 5: SAC on HalfCheetah-v4

SAC with the default fixed temperature (0.1) first exceeds an eval return of 6000 at 285K steps and reaches 10,355 (final 10,212) after 1M steps.

3.5 Automatic temperature tuning

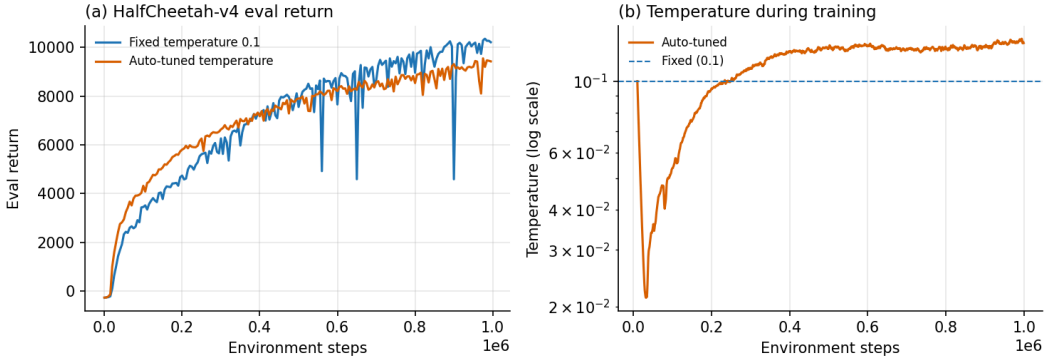


Figure 6: Fixed versus auto-tuned temperature on HalfCheetah-v4

Does auto-tuning improve or achieve comparable performance? It is comparable, slightly lower at the end. The auto-tuned run reaches 6000 a little earlier (245K versus 285K steps) and ends at 9425 (maximum 9549), versus 10,212 (maximum 10,355) for the fixed temperature. With one seed per setting, a difference of this size (about 8%) is not conclusive, and the handout notes that the fixed value 0.1 is already well tuned for this task. The useful property is that the auto-tuned run obtained this result without choosing the temperature by hand.

How does the temperature evolve? It starts at the initial value of 0.1, *decreases* quickly to about 0.04 during the first 80K steps, then *increases* slowly, and *stabilizes* at about 0.13 from roughly 400K steps onwards. It therefore ends close to the hand-tuned value of 0.1.

Why does it change in this way for HalfCheetah? The temperature is the dual variable of the constraint that the policy entropy should be at least the target, $-\dim(\mathcal{A}) = -6$. It decreases when the policy entropy is above the target and increases when it is below. At the start, the policy is close to its wide initial distribution and its entropy is far above −6, so the entropy bonus is not needed and the temperature falls. As the critic becomes accurate and the returns (and thus the Q-value differences between actions) grow into the thousands, the actor is pushed towards a nearly deterministic running gait and the entropy drops to the target. From then

on a larger temperature is needed to keep the entropy from falling below −6 against an increasingly strong Q-gradient, so the temperature rises until the two forces balance. The logged entropy confirms this: with auto-tuning it stays at about −6 (the target) for the rest of training, whereas with the fixed temperature it drifts down to about −8.

3.6 Clipped double-Q on Hopper-v4

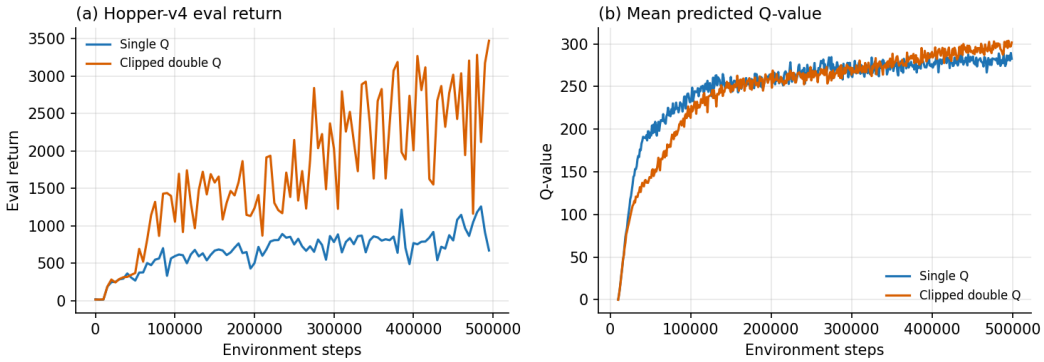


Figure 7: Single-Q versus clipped double-Q on Hopper-v4, submitted runs (seed 3)

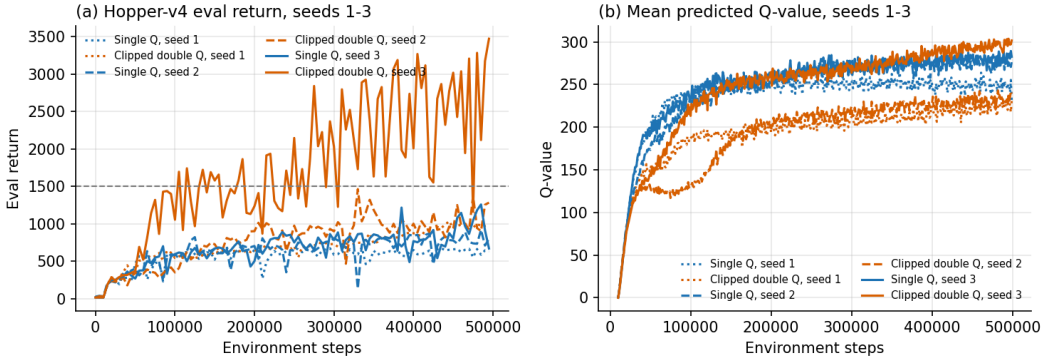


Figure 8: All three seeds

Seed	Single-Q: max / mean of last 10 evals	Clipped double-Q: max / mean of last 10 evals	Mean predicted Q at the end (single / clipped)
1	697 / 634	1034 / 976	248 / 225
2	919 / 787	1464 / 1084	275 / 234
3 (submitted)	1258 / 993	3474 / 2686	282 / 297

Which one works best? Clipped double-Q. It is better than single-Q for each of the three seeds, in both the maximum and the final return. With seed 3 it first exceeds 1500 at 105K steps, is above 1500 in 56 of the 100 evaluations, and ends at 3474. Single-Q never reaches 1500 with any seed.

About the seeds. The result depends strongly on the seed. My first run of each configuration (seed 1) plateaued at about 1000 for clipped double-Q, which misses the handout's expectation of 1500, and seed 2 peaked at 1464. Following the handout's remark that several runs may be needed, I reran both configurations with seeds 2 and 3 without changing any code or hyperparameter, and I submit the seed-3 pair. A return of about 1000 with episodes of only 200–380 steps (out of a maximum of 1000) is a known local optimum of Hopper, in which the agent hops forward a short distance and then falls; two of the three clipped double-Q seeds and all single-Q seeds stay there within 500K steps. An additional clipped double-Q run with automatic temperature tuning (seed 1, in `exp_extra/`) behaves the same way (it reaches 2321 once but ends near 980), so the plateau is not caused by the fixed temperature.

Relation to overestimation bias. With a single critic, the target $r + \gamma Q_{\phi'}(s', a')$ is evaluated at an action a' chosen by an actor that is itself trained to maximize the same critic. The actor therefore seeks out actions

where the critic's error is positive, the error is copied into the target, and it accumulates through bootstrapping. Taking the minimum of two independently initialized critics gives a pessimistic target that counteracts this. The Q-value plot shows the effect directly:

- Early in training the single-Q critic's predictions rise fastest: at 100K steps they are 226–234 for all three seeds although the policies only obtain a return of about 580. The clipped critics predict 130–213 at that point, and the highest of these (213, seed 3) belongs to a policy that already obtains 1055.
- For seeds 1 and 2, the single-Q critic ends with *higher* predicted values (248 and 275) than the clipped critic (225 and 234), while its policy obtains a *lower* real return (634 and 787 versus 976 and 1084). A higher prediction together with a lower achieved return means that the single-Q values are over-estimated relative to the clipped ones.
- For seed 3, the clipped critic's prediction is the highest of all runs (297), but here it is backed by a real return that is about 2.7 times higher than that of the single-Q run (2686 versus 993), whose prediction is nearly the same (282).

An over-estimated critic misleads the actor towards actions that only look good, which is consistent with the single-Q policies remaining at a lower return in every seed. These are three seeds per variant, so the size of the gap is uncertain, but its direction is the same in all three.

Commands

```
export WANDB_MODE=offline

# DQN
uv run src/scripts/run_dqn.py -cfg experiments/dqn/cartpole.yaml --eval_interval 2500
uv run src/scripts/run_dqn.py -cfg experiments/dqn/lunarlander.yaml
uv run src/scripts/run_dqn.py -cfg experiments/dqn/mspacman.yaml
for lr in 0.0001 0.0003 0.003; do
  uv run src/scripts/run_dqn.py -cfg experiments/dqn/lunarlander_lr$lr.yaml
done

# SAC
uv run src/scripts/run_sac.py -cfg experiments/sac/sanity_invertedpendulum.yaml
uv run src/scripts/run_sac.py -cfg experiments/sac/sanity_invertedpendulum_autotune.yaml
uv run src/scripts/run_sac.py -cfg experiments/sac/halfcheetah.yaml
uv run src/scripts/run_sac.py -cfg experiments/sac/halfcheetah_autotune.yaml
for seed in 1 2 3; do
  uv run src/scripts/run_sac.py -cfg experiments/sac/hopper_singleq.yaml --seed $seed
  uv run src/scripts/run_sac.py -cfg experiments/sac/hopper_clipq.yaml --seed $seed
done

# Figures
uv run src/scripts/plot.py
```